FRONTEND ENGINEERING PLAYBOOK

AI Study Buddy
Frontend Prompt Handbook

A step-by-step library of production-ready prompts for building the AI Study Buddy web frontend with Claude — from project planning through the final full-build master prompt. Mock-data first, API-ready by design.

BUILT WITH

React.js · Tailwind CSS · React Router · Axios · Framer Motion · Recharts

FEELS LIKE

ChatGPT · Notion · Quizlet

READ ME FIRST

How to Use This Handbook

This handbook is a sequenced set of prompts. Each section produces one artefact that the next section builds on, so the frontend comes together in the exact order a real engineer would assemble it — planning and architecture first, screens in the middle, then state, polish, mock data, and a final full-build master prompt.

1. **Work top to bottom.** Run the sections in order. Later prompts assume the outputs of earlier ones (the design tokens, the folder structure, the mock-data shapes).
2. **Copy the Detailed Prompt verbatim.** Each dark box is self-contained and already includes the project context, the tech stack, and the product feel. Paste it straight into Claude.
3. **Save every output.** Keep each result (brief, IA, tokens, specs) in your repo. Feed the relevant ones back in when a prompt refers to "earlier work".
4. **Use the five-part frame.** Every section gives you the **Objective** (why), **Expected Output** (what you get), the **Detailed Prompt** (what to run), the **Deliverables** (what to keep), and a **Checklist** (how to verify).
5. **Stay mock-first.** Everything is built on mock data behind a single swappable adapter, so wiring the real backend later changes only one layer.

> **The golden rule:** the value compounds. A strong Project Planning and Design System output early makes every screen prompt downstream sharper and more consistent. Do not skip Sections 1–4 to rush to UI.

THE PATH

Recommended Build Order

Four phases take you from a blank repo to a complete, animated, responsive frontend. Work through each phase in order; within a phase, top to bottom.

PHASE 1 · FOUNDATION

Decide what you are building and how it is shaped before drawing a pixel.

- 01 Project Planning
- 02 Information Architecture
- 03 Frontend Architecture
- 04 Design System

PHASE 2 · SCREENS

Build every page of the product against the foundation, screen by screen.

- 05 Authentication Pages
- 06 Dashboard
- 07 Notes Library
- 08 PDF Upload UI
- 09 Chat With Notes
- 10 Chat History (Sidebar)
- 11 Quiz Module
- 12 Quiz Result Analytics
- 13 Flashcards
- 14 Profile Page

PHASE 3 · SYSTEMS & POLISH

Extract reusable parts, wire state, then make it responsive and alive.

- 15 Reusable Components
- 16 State Management
- 17 Responsive Design
- 18 Animations
- 19 Mock Data

PHASE 4 · ASSEMBLY

Combine everything into one master prompt that builds the whole frontend.

- 20 Frontend Code Generation (Master Prompt)

CONTENTS

Table of Contents

01

SECTION I

Project Planning

OBJECTIVE

Establish a shared product foundation before a single screen is built — what AI Study Buddy is, who it serves, what it must do, and how users move through it. This output anchors every later prompt.

EXPECTED OUTPUT

A written product brief covering the product overview, three to four user personas, a prioritised feature list, an explicit frontend scope, and primary user journeys.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior Product Strategist. Produce a concise Product Planning Brief for AI Study Buddy with the following sections, each clearly headed:

1. PRODUCT OVERVIEW — a one-paragraph vision statement, the core problem solved, and the value proposition in a single sentence.
2. USER PERSONAS — 3 to 4 personas (e.g. undergraduate exam-crammer, lifelong self-learner, high-school student, professional upskiller). For each give: name, age, goals, frustrations, and how AI Study Buddy helps them.
3. FEATURES LIST — group features by module (Auth, Dashboard, Notes, Chat, Quiz, Flashcards, Profile) and tag each as Must-have, Should-have, or Could-have using MoSCoW.
4. FRONTEND SCOPE — an explicit in-scope vs out-of-scope table so we never build beyond the brief; note that this phase is mock-data only with no backend logic.
5. USER JOURNEYS — step-by-step journeys for: (a) first-time signup to first chat, (b) uploading a PDF and asking questions about it, (c) taking a quiz and reviewing results.

Keep it skimmable: short paragraphs, tables, and bullet points. Do not write any code.
```

DELIVERABLES

- [ ] Product overview and one-line value proposition
- [ ] 3-4 documented personas with goals and frustrations
- [ ] MoSCoW-prioritised feature list grouped by module
- [ ] In-scope vs out-of-scope table
- [ ] Three primary user-journey walkthroughs

CHECKLIST

- [ ] Vision statement is one paragraph and unambiguous
- [ ] Every persona maps to at least one core feature
- [ ] Each feature is tagged Must / Should / Could
- [ ] Out-of-scope items are explicitly listed
- [ ] All three key journeys are covered end to end

02

SECTION II

Information Architecture

OBJECTIVE

Define how the entire application is organised so navigation feels obvious. This decides the route table, the menu structure, and how pages nest before any component exists.

EXPECTED OUTPUT

A complete information-architecture document: a sitemap, the navigation structure, a page hierarchy, and the user navigation flow between screens.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior Information Architect. Design the information architecture for AI Study Buddy. Cover every screen: Landing, Login, Signup, Dashboard, Notes Library, PDF Upload, Chat With Notes, Chat History, Quiz Module, Quiz Result Analytics, Flashcards, and Profile.

Deliver the following, with no code:
1. SITEMAP — a hierarchical tree of all pages and sub-pages, distinguishing public routes (Landing, Login, Signup) from protected routes (everything after auth).
2. NAVIGATION STRUCTURE — define the primary navigation (top navbar and/or app sidebar), the items it contains, and what changes between the logged-out and logged-in states.
3. PAGE HIERARCHY — for each page list its purpose, its parent, the key sub-sections it contains, and which navigation surfaces link to it.
4. ROUTE TABLE — propose React Router paths for every screen (e.g. /dashboard, /notes, /chat/:chatId, /quiz/:quizId, /quiz/:quizId/results) and mark each route as public or protected.
5. USER NAVIGATION FLOW — describe the typical movement between screens and the redirect rules (e.g. unauthenticated users hitting a protected route are sent to Login).

Present the sitemap as an indented tree and the route table as a table.
```

DELIVERABLES

- [ ] Hierarchical sitemap (public vs protected)
- [ ] Defined primary navigation for both auth states
- [ ] Page hierarchy with purpose and parent for each screen
- [ ] React Router route table with access levels
- [ ] Navigation-flow description with redirect rules

CHECKLIST

- [ ] Every page from the brief appears in the sitemap
- [ ] Public and protected routes are clearly separated
- [ ] Route paths are RESTful and use params where needed
- [ ] Navbar/sidebar contents differ by auth state
- [ ] Redirect behaviour for protected routes is defined

03

SECTION III

Frontend Architecture

OBJECTIVE

Decide the technical skeleton of the codebase — folders, component boundaries, how state flows, and how the app talks to APIs — so the project stays maintainable as it grows.

EXPECTED OUTPUT

An architecture document with a recommended folder structure, a component breakdown, a state-management plan, and the structure of the Axios-based API layer.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Principal Frontend Architect. Define the frontend architecture for AI Study Buddy. Produce, with no implementation code (folder trees and short explanations only):

1. FOLDER STRUCTURE — a complete, opinionated src/ tree using a feature-based organisation (e.g. src/features/auth, src/features/notes, src/features/chat, src/features/quiz, src/features/flashcards), plus shared folders for components/ui, hooks, services, context or store, lib, routes, mock, and assets. Explain the responsibility of each top-level folder.
2. COMPONENT STRUCTURE — describe the component layering: page components vs feature components vs shared UI primitives. Give naming conventions and the rule for when something becomes a reusable component.
3. STATE MANAGEMENT PLAN — recommend an approach (React Context + hooks, or a lightweight store) and map which state is local, which is shared, and which is server-cache. Define the slices needed: auth, notes, chat, quiz, flashcards, ui.
4. API LAYER STRUCTURE — design a centralised Axios client (base instance, interceptors for auth tokens and errors) and per-feature service modules (authService, notesService, chatService, quizService). Specify that every service currently returns mock data through a single swappable adapter so the real API can be dropped in later.

Present folder structures as indented trees and keep explanations tight.
```

DELIVERABLES

- [ ] Feature-based src/ folder tree with responsibilities
- [ ] Component layering and naming conventions
- [ ] State-management plan mapping each state slice
- [ ] Centralised Axios client with interceptor design
- [ ] Per-feature service modules with a swappable mock adapter

CHECKLIST

- [ ] Folder tree covers every feature in the brief
- [ ] Clear rule separates pages, features, and UI primitives
- [ ] Each state slice is assigned an owner location
- [ ] Axios client centralises auth and error handling
- [ ] Mock-to-real API swap path is a single seam

04

SECTION IV

Design System

OBJECTIVE

Lock the visual language once, in tokens, so every screen is consistent and the Notion-clean / Quizlet-playful / ChatGPT-calm blend is intentional rather than accidental.

EXPECTED OUTPUT

A design-system specification: colour palette, typography scale, spacing system, base component specs, and responsive breakpoints — all expressed as reusable tokens.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior UI/UX Designer and Design Systems Lead. Define a complete design system for AI Study Buddy, expressed as tokens ready to drop into a Tailwind config. Deliver, with no component code:

1. COLOURS — a primary brand colour and full tint/shade scale, a secondary/accent colour, semantic colours (success, warning, error, info), neutral greys, and both light and dark surface colours. Give hex values and the intended usage of each.
2. TYPOGRAPHY — choose a primary UI font and a reading font, define a type scale (display, h1-h4, body, small, caption) with size / weight / line-height, and rules for headings vs long-form notes.
3. SPACING — a consistent spacing scale (e.g. 4px base) and how it maps to padding, gaps, and section rhythm; define container max-widths.
4. COMPONENTS — token-level specs for core primitives: Button (variants + sizes + states), Input, Card, Badge, Modal, Toast, Avatar, Tabs. Describe radius, shadow, and border conventions.
5. RESPONSIVE BREAKPOINTS — define sm / md / lg / xl breakpoints and the layout intent at each.

Format colours and tokens as tables. Make the palette modern, friendly, and study-focused.
```

DELIVERABLES

- [ ] Full colour palette with scales and semantic colours
- [ ] Typography scale with fonts, sizes, and line-heights
- [ ] Spacing scale and container widths
- [ ] Token specs for core component primitives
- [ ] Responsive breakpoint definitions with layout intent

CHECKLIST

- [ ] Primary palette includes a usable tint/shade scale
- [ ] Semantic colours defined for success/warning/error/info
- [ ] Type scale covers display through caption
- [ ] Spacing scale is consistent and base-unit driven
- [ ] Light and dark surface colours both specified

05

SECTION V

Authentication Pages

OBJECTIVE

Build the front door of the app — Login and Signup — with trustworthy validation, error handling, and loading feedback, all running on mock auth for now.

EXPECTED OUTPUT

Specifications and build prompts for the Login and Signup pages, including form validation rules, error states, and loading states, wired to a mock auth service.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior React Developer. Specify the Authentication pages for AI Study Buddy. Cover:

1. LOGIN PAGE — layout (centred card on a branded split or gradient background), email + password fields, 'remember me', a link to Signup, and a primary submit button. Describe the desktop and mobile layouts.
2. SIGNUP PAGE — name, email, password, confirm-password, terms checkbox, and a link back to Login. Include a live password-strength hint.
3. FORM VALIDATION — define client-side rules for every field (required, email format, min password length, matching confirmation) and whether validation fires on blur or on submit. Recommend react-hook-form patterns conceptually.
4. ERROR STATES — inline field errors, a form-level error banner for failed mock login, and clear, human-readable messages.
5. LOADING STATES — disabled submit + spinner during the simulated async call, and a skeleton or transition for redirect after success.

The auth call should hit a mock authService that resolves/rejects after a short delay and stores a fake token in auth state. Describe the components and states needed; do not write full code unless asked in a later prompt.
```

DELIVERABLES

- Login page layout and field spec
- Signup page layout with password-strength hint
- Per-field validation rules and timing
- Inline and form-level error-state designs
- Loading and post-success transition states

CHECKLIST

- [ ] Both pages have responsive desktop + mobile layouts
- [ ] Every field has explicit validation rules
- [ ] Failed mock login shows a clear banner message
- [ ] Submit button disables and shows a spinner while pending
- [ ] Successful auth stores a token and redirects

06

SECTION VI

Dashboard

OBJECTIVE

Give returning users a calm, informative home base that summarises their study activity and offers fast paths into the core tools.

EXPECTED OUTPUT

A build prompt for the Dashboard featuring statistics cards, charts, a recent-activity feed, quick actions, and considered empty states.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior React Developer and UI/UX Designer. Specify the Dashboard for AI Study Buddy. It is the first screen after login and should feel like a Notion home with light analytics. Cover:

1. STATISTICS CARDS — a responsive row of stat cards: total notes, quizzes taken, average score, flashcards mastered, study streak. Each card shows a value, a label, an icon, and a small trend indicator.
2. CHARTS — using Recharts, a study-activity line/area chart over time and a quiz-performance bar chart by subject. Specify axes, tooltips, and responsiveness.
3. RECENT ACTIVITY — a feed of the latest actions (uploaded a PDF, finished a quiz, created flashcards) with icon, label, and relative timestamp.
4. QUICK ACTIONS — prominent buttons/cards for Upload PDF, Start Chat, Take a Quiz, and Review Flashcards that route to the relevant screens.
5. EMPTY STATES — friendly first-run states for a brand-new user with no data, each with an illustration placeholder, a one-line message, and a call-to-action.

All data comes from mock sources. Describe the grid layout at desktop, tablet, and mobile.
```

DELIVERABLES

- [ ] Responsive statistics-card row spec
- [ ] Two Recharts visualisations (activity + performance)
- [ ] Recent-activity feed with relative timestamps
- [ ] Quick-action shortcuts routing to core tools
- [ ] Empty-state designs for new users

CHECKLIST

- [ ] Stat cards reflow cleanly across breakpoints
- [ ] Charts have tooltips and are responsive
- [ ] Activity feed shows icon, label, and time
- [ ] Quick actions deep-link to the right routes
- [ ] Every data area has a defined empty state

07

SECTION VII

Notes Library

OBJECTIVE

Let users browse, find, and manage their uploaded study material in a tidy, Notion-like library that scales from a handful of notes to hundreds.

EXPECTED OUTPUT

A build prompt for the Notes Library with a notes grid, search, filters, an upload entry point, and delete actions.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior React Developer. Specify the Notes Library for AI Study Buddy. Cover:

1. NOTES GRID — a responsive grid of note cards, each showing a title, source type (PDF, pasted text), subject tag, page/word count, last-opened date, and a thumbnail or file icon. Support a grid/list toggle.
2. SEARCH — an instant client-side search over title and tags with a clear-input control and a no-results state.
3. FILTERS — filter by subject/tag and by source type, plus sort options (recent, A-Z, most studied). Filters combine with search.
4. UPLOAD AREA — a visible entry point (button + small dropzone) that opens the PDF Upload flow specified in the next section.
5. DELETE ACTIONS — a per-card menu with delete, guarded by a confirmation modal and an undo toast.

All notes come from mock data. Include loading skeletons for the grid and an empty-library state. Describe responsive behaviour from desktop multi-column down to a single mobile column.
```

DELIVERABLES

- [ ] Responsive note-card grid with grid/list toggle
- [ ] Instant search with no-results handling
- [ ] Combinable filters and sort options
- [ ] Upload entry point into the PDF flow
- [ ] Guarded delete with confirmation and undo

CHECKLIST

- [ ] Cards show all key metadata fields
- [ ] Search and filters work together
- [ ] Grid has loading skeletons and an empty state
- [ ] Delete requires confirmation
- [ ] Layout collapses to one column on mobile

08

SECTION VIII

PDF Upload UI

OBJECTIVE

Make adding study material effortless and reassuring with a drag-and-drop uploader that shows honest progress and handles failures gracefully.

EXPECTED OUTPUT

A build prompt for the PDF upload experience: drag-and-drop, progress tracking, success states, and error states, simulated against mock upload logic.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior React Developer. Specify the PDF Upload UI for AI Study Buddy, usable both as a modal and as a standalone area. Cover:

1. DRAG & DROP UPLOAD — a dropzone that responds to drag-enter/over/leave states, plus a click-to-browse fallback. Validate file type (PDF) and a max file size, with clear messaging when invalid.
2. PROGRESS TRACKING — a per-file progress bar with percentage, support for multiple files queued at once, and a cancel control. Simulate progress with a mock timer that increments to 100%.
3. SUCCESS STATES — a per-file success tick, a summary when all files finish, and an auto-route or CTA into the new note / Chat With Notes.
4. ERROR STATES — handle invalid type, oversize, and a simulated random upload failure with a retry action per file.

Everything runs on mock upload logic (no real network). Describe accessibility (keyboard focus on the dropzone, ARIA live region for progress) and the mobile layout.
```

DELIVERABLES

- [ ] Drag-and-drop dropzone with click fallback
- [ ] File-type and size validation with messaging
- [ ] Multi-file progress bars with cancel
- [ ] Per-file and summary success states
- [ ] Error states with per-file retry

CHECKLIST

- [ ] Dropzone reacts to all drag states
- [ ] Invalid type/size is rejected with a clear message
- [ ] Progress simulates to 100% and is cancellable
- [ ] Success offers a path into the new note
- [ ] Failures can be retried individually

09

SECTION IX

Chat With Notes

OBJECTIVE

Deliver the signature feature: a ChatGPT-style conversation grounded in the user's notes, that feels fast, alive, and easy to read.

EXPECTED OUTPUT

A build prompt for the chat experience: ChatGPT-like layout, chat window, message bubbles, input area, AI response area, typing indicator, and auto-scroll.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior React Developer. Specify the Chat With Notes screen for AI Study Buddy — the most ChatGPT-like surface in the app. Cover:

1. CHATGPT-LIKE LAYOUT — a two-pane layout: a chat history sidebar (detailed in the next section) on the left and the active conversation on the right, with a header showing which note(s) the chat is grounded in.
2. CHAT WINDOW — a scrollable, full-height message list with comfortable reading width and clear separation between turns.
3. MESSAGE BUBBLES — distinct styles for user vs AI messages, with avatar/role label, timestamp, and per-message actions (copy, regenerate). AI messages should render markdown (headings, lists, code, bold).
4. INPUT AREA — a pinned, auto-growing textarea with send button, Enter-to-send / Shift+Enter newline, a disabled state while awaiting a reply, and an attach-note control.
5. AI RESPONSE AREA — a streamed-feeling response (reveal text progressively from mock data) and a source-citation chip referencing the note.
6. TYPING INDICATOR — an animated 'AI is thinking' indicator (Framer Motion) shown before the response begins.
7. AUTO SCROLL — keep the view pinned to the newest message, with a 'scroll to bottom' button when the user has scrolled up.

Responses come from a mock chat service that returns canned answers after a delay. Specify the mobile layout where the sidebar collapses into a drawer.
```

DELIVERABLES

- [ ] Two-pane ChatGPT-style layout with grounded header
- [ ] Scrollable chat window with readable width
- [ ] User/AI bubbles with markdown and per-message actions
- [ ] Auto-growing input with keyboard shortcuts
- [ ] Typing indicator and progressive response reveal
- [ ] Auto-scroll with a scroll-to-bottom control

CHECKLIST

- [ ] User and AI messages are visually distinct
- [ ] AI messages render markdown correctly
- [ ] Input supports Enter-send and Shift+Enter newline
- [ ] Typing indicator appears before each response
- [ ] View auto-scrolls and offers scroll-to-bottom
- [ ] Sidebar collapses to a drawer on mobile

10

SECTION X

Chat History (Sidebar)

OBJECTIVE

Let users manage many conversations the way ChatGPT does — create, find, rename, delete, and resume chats without friction.

EXPECTED OUTPUT

A build prompt for the chat history sidebar: new chat, rename, delete, search, and continue an old chat.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior React Developer. Specify the Chat History sidebar for AI Study Buddy. Cover:

1. CHAT SIDEBAR — a scrollable list of past conversations grouped by recency (Today, Yesterday, Previous 7 days, Older), each item showing the chat title, a snippet, and the linked note. Highlight the active chat.
2. NEW CHAT — a prominent 'New chat' button that creates a fresh conversation and focuses the input.
3. RENAME CHAT — inline rename via a context menu or double-click, with auto-title from the first message as the default.
4. DELETE CHAT — delete from the context menu, guarded by a confirmation and an undo toast.
5. SEARCH CHATS — a search box filtering the list by title and message content with a no-results state.
6. CONTINUE OLD CHAT — selecting a chat loads its full message history into the chat window and updates the route to /chat/:chatId.

All chats come from mock data and persist in chat state for the session. Specify the collapsed drawer behaviour on mobile and keyboard accessibility for the list.
```

DELIVERABLES

- [ ] Recency-grouped chat list with active highlight
- [ ] New-chat action that focuses the input
- [ ] Inline rename with auto-title default
- [ ] Guarded delete with undo
- [ ] Search across titles and message content
- [ ] Resume-chat loading full history by route

CHECKLIST

- [ ] Chats are grouped by recency
- [ ] New chat creates and focuses a fresh conversation
- [ ] Rename and delete are reachable from a menu
- [ ] Search filters by title and content
- [ ] Selecting a chat restores its full history
- [ ] Sidebar works as a mobile drawer

11

SECTION XI

Quiz Module

OBJECTIVE

Turn notes into an engaging, Quizlet-style multiple-choice test with clear progress, a timer, and a confident submission flow.

EXPECTED OUTPUT

A build prompt for the quiz-taking experience: MCQ interface, timer, progress bar, navigation, and submission flow.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior React Developer. Specify the Quiz Module for AI Study Buddy. Cover:

1. MCQ INTERFACE — one question per screen (or a clean scroll), the question text, four selectable option cards with clear selected/hover/focus states, and support for single-correct answers. Allow flagging a question for review.
2. TIMER — an overall quiz countdown shown in the header that auto-submits at zero, plus an optional per-question time hint. Warn visually in the final seconds.
3. PROGRESS BAR — a progress indicator showing answered vs total and current position, animated with Framer Motion.
4. NAVIGATION — Previous / Next controls and a question-number grid (a navigator) to jump around, marking answered, unanswered, and flagged questions.
5. SUBMISSION FLOW — a submit button that opens a review summary (how many answered/skipped), a confirmation modal, then routes to the Quiz Result Analytics screen.

Questions come from mock data. Persist answers in quiz state so navigation never loses a selection. Specify the mobile layout and keyboard navigation between options.
```

DELIVERABLES

- [ ] Single-question MCQ interface with option states
- [ ] Countdown timer with auto-submit
- [ ] Animated progress bar
- [ ] Prev/Next plus a jump-to navigator grid
- [ ] Review summary and confirmed submission flow

CHECKLIST

- [ ] Option cards show selected/hover/focus states
- [ ] Timer auto-submits and warns near zero
- [ ] Progress reflects answered vs total
- [ ] Navigator marks answered/flagged questions
- [ ] Submission confirms before scoring

12

SECTION XII

Quiz Result Analytics

OBJECTIVE

Help users learn from each attempt with a clear score summary, performance insights, charts, and a full answer review.

EXPECTED OUTPUT

A build prompt for the results screen: score summary, performance metrics, charts, and answer review.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior React Developer and Data-Viz Designer. Specify the Quiz Result Analytics screen for AI Study Buddy. Cover:

1. SCORE SUMMARY — a hero area with the final score (e.g. an animated percentage ring), pass/fail or grade band, correct vs incorrect counts, and time taken.
2. PERFORMANCE METRICS — breakdowns by topic/subject and by difficulty, accuracy percentage, and a comparison against the user's previous attempts.
3. CHARTS — using Recharts: a per-topic accuracy bar chart and a trend line of scores across past attempts, both responsive with tooltips.
4. REVIEW ANSWERS — a per-question review list showing the user's answer, the correct answer, right/wrong status, and an explanation, with a filter to show only incorrect questions.

Add primary actions: Retake quiz, Make flashcards from missed questions, and Back to dashboard. All data is mock. Describe the responsive layout and the celebratory micro-animation on a high score.
```

DELIVERABLES

- [ ] Animated score-summary hero
- [ ] Topic and difficulty performance breakdowns
- [ ] Two responsive Recharts visualisations
- [ ] Filterable per-question answer review
- [ ] Follow-up actions (retake / flashcards / dashboard)

CHECKLIST

- [ ] Score, counts, and time are all shown
- [ ] Metrics break down by topic and difficulty
- [ ] Charts are responsive with tooltips
- [ ] Review shows correct answer and explanation
- [ ] Incorrect-only filter works

13

SECTION XIII

Flashcards

OBJECTIVE

Provide a satisfying, Quizlet-style flashcard study mode with a smooth flip, easy navigation, and shuffle for varied practice.

EXPECTED OUTPUT

A build prompt for flashcards: card layout, flip animation, previous/next navigation, and a shuffle feature.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior React Developer. Specify the Flashcards study mode for AI Study Buddy. Cover:

1. FLASHCARD LAYOUT — a large centred card showing the term/question on the front and the definition/answer on the back, a deck title, and a position counter (e.g. 4 / 20). Include a self-grading control (Got it / Review again) that feeds a mastery count.
2. FLIP ANIMATION — a smooth 3D flip on click/tap/Space using Framer Motion, with correct backface handling and no layout jump.
3. PREVIOUS / NEXT NAVIGATION — arrow buttons plus left/right keyboard arrows and swipe gestures on mobile, resetting the card to its front face on change.
4. SHUFFLE FEATURE — a shuffle toggle that randomises deck order, plus a restart and a 'study only unmastered' filter. Show a deck-complete summary at the end.

Cards come from mock decks. Specify accessibility (focus management, ARIA for flip state) and the responsive sizing of the card.
```

DELIVERABLES

- [ ] Centred flashcard with front/back and counter
- [ ] Smooth 3D flip animation
- [ ] Prev/next via buttons, keys, and swipe
- [ ] Shuffle, restart, and unmastered-only filter
- [ ] Deck-complete summary with mastery stats

CHECKLIST

- [ ] Card flips smoothly without layout jump
- [ ] Front/back content is correct after flip
- [ ] Navigation resets the card face
- [ ] Shuffle randomises order
- [ ] Deck completion shows a summary

14

SECTION XIV

Profile Page

OBJECTIVE

Give users a single place to see who they are, review their progress, adjust preferences, and manage their account safely.

EXPECTED OUTPUT

A build prompt for the profile page: user information, statistics, settings, and account management.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior React Developer. Specify the Profile page for AI Study Buddy. Cover:

1. USER INFORMATION — avatar (with upload/change), display name, email, join date, and an edit-profile mode with inline validation.
2. STATISTICS — a summary of lifetime activity: notes, quizzes taken, average score, flashcards mastered, and current streak, reusing the dashboard stat-card component.
3. SETTINGS — preferences such as theme (light/dark/system), notification toggles, default quiz length, and language, each persisted to mock user state.
4. ACCOUNT MANAGEMENT — change password, log out, and a guarded delete-account flow with a confirmation modal.

Use a tabbed or sectioned layout (Profile / Stats / Settings / Account). All data is mock. Specify the responsive layout and which actions require confirmation.
```

DELIVERABLES

- [ ] Editable user-information section with avatar
- [ ] Reused statistics cards for lifetime activity
- [ ] Persisted settings (theme, notifications, defaults)
- [ ] Account management (password, logout, delete)
- [ ] Tabbed/sectioned responsive layout

CHECKLIST

- [ ] Profile edit mode validates inputs
- [ ] Stats reuse the shared stat-card component
- [ ] Settings changes persist to mock state
- [ ] Delete account is guarded by confirmation
- [ ] Layout adapts across breakpoints

15

SECTION XV

Reusable Components

OBJECTIVE

Extract the recurring building blocks into a documented component library so every feature is assembled from consistent, tested primitives.

EXPECTED OUTPUT

Build prompts for the shared component library: Navbar, Sidebar, Stats Card, PDF Card, Chat components, Quiz components, and Flashcard components.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Principal Frontend Architect. Design a reusable component library for AI Study Buddy. For each component below, specify its purpose, props/API, variants, states, and where it is used — no full code, just clear contracts:

1. NAVBAR — top bar with logo, primary links, search, and a user menu; differs by auth state.
2. SIDEBAR — collapsible app navigation / chat-history container with active states.
3. STATS CARD — value, label, icon, trend; used on Dashboard and Profile.
4. PDF CARD — note card with title, type, tags, metadata, thumbnail, and a menu.
5. CHAT COMPONENTS — MessageBubble, TypingIndicator, ChatInput, ChatHeader.
6. QUIZ COMPONENTS — QuestionCard, OptionItem, QuizTimer, ProgressBar, QuestionNavigator.
7. FLASHCARD COMPONENTS — FlashCard, DeckControls, MasteryBadge.

For each, list the props, the variants, the interactive states (default/hover/focus/active/disabled/loading), and accessibility notes. Present each component as a short spec block.
```

DELIVERABLES

- [ ] Navbar and Sidebar contracts with auth/active states
- [ ] Stats Card and PDF Card prop APIs
- [ ] Chat component set (bubble, indicator, input, header)
- [ ] Quiz component set (question, option, timer, progress, navigator)
- [ ] Flashcard component set with mastery badge

CHECKLIST

- [ ] Every component lists props and variants
- [ ] Interactive states are enumerated
- [ ] Each component notes where it is reused
- [ ] Accessibility notes are included
- [ ] Naming is consistent with the architecture

16

SECTION XVI

State Management

OBJECTIVE

Specify how each domain's data is stored, updated, and shared so features stay in sync and the mock-to-API swap is painless.

EXPECTED OUTPUT

Build prompts for the state layer: auth state, notes state, chat state, quiz state, and flashcard state.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior React Developer. Design the state management for AI Study Buddy using the approach chosen in the architecture phase (Context + hooks or a lightweight store). For each slice below specify: the shape of the state, the actions/methods, the derived selectors, and which mock service feeds it. No full implementation, just clear contracts.

1. AUTH STATE — current user, token, isAuthenticated, loading/error; actions login, signup, logout, loadSession.
2. NOTES STATE — notes list, filters, search term, selected note; actions fetchNotes, addNote, deleteNote, setFilter.
3. CHAT STATE — chats list, activeChatId, messages by chat, sending status; actions newChat, sendMessage, renameChat, deleteChat, loadChat.
4. QUIZ STATE — current quiz, answers map, current index, timer, result; actions startQuiz, answer, next/prev, submitQuiz.
5. FLASHCARD STATE — decks, active deck, current index, flipped, mastery map; actions loadDeck, flip, next/prev, shuffle, markMastered.

Note for each slice which parts are server-cache (swappable to API) vs pure UI state. Present each slice as a structured spec.
```

DELIVERABLES

- [ ] Auth slice contract with session handling
- [ ] Notes slice with filters and search
- [ ] Chat slice with per-chat message storage
- [ ] Quiz slice with answers, timer, and result
- [ ] Flashcard slice with mastery tracking

CHECKLIST

- [ ] Each slice defines shape, actions, and selectors
- [ ] Mock service feeding each slice is named
- [ ] Server-cache vs UI state is distinguished
- [ ] Loading and error states exist per slice
- [ ] Actions cover every user interaction

17

SECTION XVII

Responsive Design

OBJECTIVE

Guarantee the app works beautifully from large desktops down to phones, with special care for the most layout-heavy screens.

EXPECTED OUTPUT

Build prompts defining responsive behaviour for desktop, tablet, and mobile, with focused attention on the Dashboard, Chat, and Quiz pages.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior UI/UX Designer. Define the responsive strategy for AI Study Buddy across the breakpoints set in the design system. Cover:

1. DESKTOP (lg/xl) — multi-column layouts, persistent sidebars, and full data density.
2. TABLET (md) — how columns collapse, when sidebars become toggleable, and touch-target sizing.
3. MOBILE (sm) — single-column stacking, bottom or drawer navigation, and thumb-friendly controls.

Then give detailed, screen-specific guidance for the three hardest screens:
- DASHBOARD: how the stat-card row, charts, and quick actions reflow at each breakpoint.
- CHAT PAGE: how the history sidebar becomes a drawer, how the input stays pinned above the mobile keyboard, and how message width adapts.
- QUIZ PAGE: how the question, options, timer, and navigator rearrange on small screens without losing the timer or progress.

Specify a mobile-first Tailwind approach and list the breakpoints used. No code, just rules and layout descriptions.
```

DELIVERABLES

- [ ] Breakpoint-by-breakpoint layout rules
- [ ] Sidebar/drawer behaviour across sizes
- [ ] Dashboard reflow specification
- [ ] Chat page mobile keyboard + drawer handling
- [ ] Quiz page small-screen rearrangement

CHECKLIST

- [ ] Rules given for desktop, tablet, and mobile
- [ ] Sidebars convert to drawers on small screens
- [ ] Chat input stays pinned above the keyboard
- [ ] Quiz timer and progress remain visible on mobile
- [ ] Approach is explicitly mobile-first

18

SECTION XVIII

Animations

OBJECTIVE

Add motion that guides attention and adds delight without slowing the app, using Framer Motion consistently across the product.

EXPECTED OUTPUT

Build prompts for animations: page transitions, card animations, the flashcard flip, chat effects, and loading effects.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior Frontend Developer specialising in motion. Define the animation system for AI Study Buddy using Framer Motion. Keep motion purposeful, fast (mostly 150-300ms), and respectful of prefers-reduced-motion. Specify, for each, the trigger, the properties animated, easing, and duration:

1. PAGE TRANSITIONS — route-change enter/exit transitions (subtle fade + slide) via AnimatePresence.
2. CARD ANIMATIONS — staggered entrance for grids (notes, stat cards), and hover lift on interactive cards.
3. FLASHCARD FLIP — the 3D flip timing and easing, kept consistent with the Flashcards section.
4. CHAT EFFECTS — message bubbles animating in, the typing-indicator dots, and the progressive response reveal.
5. LOADING EFFECTS — skeleton shimmer, button spinners, and progress-bar fills.

Provide a small set of shared motion variants/tokens (durations and easings) to reuse everywhere. Note the reduced-motion fallbacks. No full code, just the motion spec.
```

DELIVERABLES

- [ ] Route page-transition spec via AnimatePresence
- [ ] Card entrance stagger and hover lift
- [ ] Flashcard flip timing aligned with Section 13
- [ ] Chat message + typing + reveal animations
- [ ] Loading shimmer, spinners, and progress fills

CHECKLIST

- [ ] Shared motion tokens (duration/easing) defined
- [ ] prefers-reduced-motion fallbacks specified
- [ ] Page transitions use AnimatePresence
- [ ] Chat typing and reveal animations described
- [ ] Loading states have consistent motion

19

SECTION XIX

Mock Data

OBJECTIVE

Create realistic, richly-shaped mock data so every screen can be built, demoed, and tested before any backend exists — and swapped for the API cleanly.

EXPECTED OUTPUT

Build prompts for mock data: users, notes, chats, messages, quizzes, and flashcards, with shapes that match the eventual API.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Senior React Developer. Design the mock-data layer for AI Study Buddy. For each entity below define a clear, API-realistic schema (field names, types, and example records) and the volume of seed records needed to make screens look alive. Keep IDs and relationships consistent across entities so a user owns notes, notes power chats and quizzes, etc.

1. USERS — id, name, email, avatar, joinDate, preferences, and aggregate stats.
2. NOTES — id, ownerId, title, sourceType, subject/tags, pageCount, createdAt, lastOpenedAt, thumbnail.
3. CHATS — id, ownerId, title, linkedNoteId, createdAt, updatedAt.
4. MESSAGES — id, chatId, role (user/ai), content (markdown), createdAt, optional citations.
5. QUIZZES — id, noteId, title, questions[] (with options, correctIndex, explanation, topic, difficulty), plus attempt records with answers, score, and timing.
6. FLASHCARDS — deck id, noteId, title, cards[] (front, back, mastery state).

Also specify a single mock service/adapter pattern that returns this data with simulated latency, so swapping to Axios calls later touches only that adapter. Present schemas as tables with sample rows.
```

DELIVERABLES

- [ ] API-realistic schemas for all six entities
- [ ] Consistent IDs and relationships across entities
- [ ] Enough seed records to populate every screen
- [ ] A latency-simulating mock service/adapter
- [ ] Sample rows for each entity

CHECKLIST

- [ ] Every entity has a typed schema
- [ ] Relationships (owner/note/chat/quiz) are consistent
- [ ] Seed volume fills grids, lists, and charts
- [ ] Mock adapter simulates latency
- [ ] Adapter is the single swap point to the API

20

SECTION XX

Frontend Code Generation (Master Prompt)

OBJECTIVE

Combine every prior output into one authoritative build instruction that assembles the complete, production-quality AI Study Buddy frontend on mock data.

EXPECTED OUTPUT

A single master prompt that directs Claude to build the entire frontend using all previous sections as inputs.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
You are a Principal Frontend Architect and Senior React Developer building the AI Study Buddy web application. The stack is React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts. The frontend is built first with mock data and wired to real APIs later. The product should feel like a blend of ChatGPT (conversational), Notion (clean, document-centric) and Quizlet (playful study tools). Prioritise accessibility, responsiveness, and reusable components.

Act as a Principal Frontend Architect and Senior React Developer. Build the complete AI Study Buddy frontend as a production-quality React application running entirely on mock data, ready for later API integration. Use every artefact produced earlier in this handbook as the source of truth:

- The Project Planning brief, Information Architecture, and Frontend Architecture (folder structure, components, state, API layer).
- The Design System tokens (colours, typography, spacing, breakpoints) configured into Tailwind.
- The per-screen specs: Landing, Login, Signup, Dashboard, Notes Library, PDF Upload, Chat With Notes, Chat History, Quiz Module, Quiz Result Analytics, Flashcards, and Profile.
- The Reusable Components library, State Management slices, Responsive rules, Animations, and Mock Data layer.

Requirements:
1. Use React.js, Tailwind CSS, React Router, Axios, Framer Motion, and Recharts.
2. Implement the full route table with public and protected routes and a route guard.
3. Wire all data through the mock service/adapter so swapping to real Axios endpoints touches only that layer.
4. Build every screen and every reusable component to the specs, fully responsive and animated.
5. Include loading, empty, and error states everywhere they were specified.
6. Follow the design tokens exactly; ensure accessibility and keyboard support.

Deliver the project as a clear, well-organised set of files following the agreed folder structure, with a short README explaining how to run it and where the mock-to-API swap happens. Build incrementally and keep components small and reusable.
```

DELIVERABLES

- A complete, runnable mock-data React frontend
- Full routing with public/protected guards
- Every screen and reusable component implemented
- All loading/empty/error states in place
- README covering run steps and the API-swap seam

CHECKLIST

- [ ] All twelve screens are built to spec
- [ ] Design tokens are applied consistently
- [ ] State slices and mock adapter are wired in
- [ ] App is responsive and animated throughout
- [ ] Mock-to-API swap is isolated to one layer

End of Handbook · AI Study Buddy Frontend Prompt Handbook · Edition 1.0